PRD — EscolaAgenda

Documento de requisitos do produto, montado a partir do levantamento técnico do repositório `projetoIntegrador` e da leitura direta do código do protótipo (`prototipo/`). Serve como base de alinhamento para a equipe antes de iniciar a construção do backend.
Data: 2026-09-08 (v3 — stack técnica fechada com a equipe + ponto de analytics adicionado)

---

1. O que é o produto

EscolaAgenda é um sistema de gestão de agenda escolar para três perfis — Professor, Direção e Coordenação — com o calendário/agenda de eventos como núcleo do produto (reuniões, avaliações, planejamento, tutorias, eventos institucionais), apoiado por um fluxo de planos de aula (envio pelo professor → validação pela direção) e um sistema de notificações.

O frontend (protótipo em React) já está bem avançado visualmente e parcialmente conectado a dados reais via Supabase. **Não existe hoje nenhum backend próprio** — o que vamos construir.

---

2. Stack técnica (decidida)

Camada	Escolha
Frontend	React 18 + TypeScript + Vite + Tailwind CSS v4 + React Router 7 + Motion (Framer Motion) + Radix UI — já implementado em `prototipo/` , mantém-se
Backend	**Node.js + Express + TypeScript**
ORM	**Prisma**
Banco de dados	**PostgreSQL gerenciado via Neon** (free tier)
Autenticação	**JWT emitido pelo próprio backend**, validado contra `usuarios.senha_hash`
Armazenamento de arquivos	**Supabase Storage** (usado só como bucket de arquivos — sem Auth nem tabelas do Supabase; DB e lógica seguem 100% no backend novo)
Hospedagem do backend	**Render**

Observações sobre as decisões

- **Autenticação (JWT próprio):** o backend expõe um endpoint de login que verifica e-mail/senha contra `usuarios` e devolve um JWT. O frontend passa a guardar esse token e enviá-lo nas chamadas à API. Login só deve suceder para usuários com `status = ativo` .

- **Supabase Storage:** aproveita o projeto Supabase que a equipe já tem configurado (o frontend já previa um bucket, `avatarsBucket`, em `lib/supabase.ts`). Evita montar credenciais/infra nova só para arquivos, e o free tier (1GB) é suficiente para o volume esperado do projeto.
- **Render:** atenção que o filesystem dos serviços web do Render (planos free/starter) é **efêmero** — qualquer arquivo salvo em disco local é perdido a cada deploy ou reinício. É mais um motivo para os anexos irem direto para o Supabase Storage, nunca para disco do próprio backend.

---

3. Perfis de usuário (papéis)

O schema de banco já modela 6 papéis (`papel_usuario`): `professor`, `diretor`, `coordenador`, `vice_diretor`, `secretario`, `administrador`.

O frontend, na tela de seleção de perfil, hoje só oferece 3 opções ao usuário: **Professor**, **Direção** e **Coordenação**. Fica em aberto para a equipe se os papéis administrativos (`vice_diretor`, `secretario`, `administrador`) entram no MVP ou ficam para depois — hoje eles existem só no banco, sem nenhuma tela associada.

Importante: a seleção de perfil no protótipo é **apenas cosmética** — não é persistida em lugar nenhum (nem contexto React, nem localStorage). Ao continuar, o usuário só é redirecionado para `/dashboard`; as rotas `/dashboard/agenda`, `/dashboard/professor`, `/dashboard/direcao` etc. são todas públicas e acessíveis diretamente por URL, sem checagem de papel. Com a autenticação JWT entrando no backend, o frontend também precisa ganhar guarda de rotas por papel — hoje não existe nenhuma.

---

4. Módulos e regras de negócio extraídas do protótipo

4.1 Autenticação e login (LoginPage, ProfileSelectionPage)

- **Estado atual: 100% simulado.** O formulário de login não valida credenciais — só espera 1 segundo (`setTimeout`) e navega para `/selecionar-perfil`, independente do que foi digitado. O botão "Login Institucional" faz a mesma coisa sem nenhum input.
- Não há sessão, token, ou usuário autenticado em lugar nenhum do app.
- **Regra de negócio a implementar no backend:** login por e-mail/senha contra `usuarios.senha_hash`; e-mail é único por usuário (`usuarios.email UNIQUE`); usuário tem `status` (`ativo` / `inativo` / `bloqueado` / `pendente`) — login deve falhar para status diferente de `ativo`.
- Recuperação de senha (RecoverPasswordPage) também é só UI, sem envio de e-mail real.

4.2 Agenda Escolar (AgendaPage) — núcleo do produto

Este é o módulo mais completo e o único, junto com a leitura de eventos em outras telas, que já fala de fato com um banco (Supabase) hoje.

- **Categorias de evento usadas na criação/exibição:** `reuniao`, `avaliacao`, `planejamento`, `tutoria`, `pais` (mapeadas para cores fixas: azul, roxo, laranja, ciano, verde). O enum do banco (`categoria_evento`) tem mais valores (`aula`, `institucional`, `conselho_classe`, `formacao`, `apoio_presencial`, `pausa`, `outro`) que a agenda escolar não usa diretamente — alguns desses aparecem em outras telas.
- **3 modos de visualização:** mensal, semanal (padrão), diário — todos client-side, sem paginação, sobre os mesmos dados carregados.
- **Busca:** filtro por texto livre que varre título, descrição, local e participantes simultaneamente.
- **Criação de evento — validação atual:** só o título é obrigatório. Data, categoria, horários, local e participantes têm valores default (categoria = `reuniao`, hoje como data, `09:00` – `10:00`). **Não há validação de que o horário de término seja depois do início**, nem checagem de conflito de horário/sala.
- **Participantes** são digitados como texto livre separado por vírgula (não há seleção de usuários/turmas cadastrados na tela) — no banco isso vira `evento_participantes.nome_livre`, não uma referência real a `usuarios` / `turmas`.
- Toda gravação depende de duas variáveis de ambiente hoje amarradas ao Supabase (`VITE_SUPABASE_DEFAULT_CALENDAR_ID`, `VITE_SUPABASE_DEFAULT_USER_ID`) — todo evento criado hoje é sempre associado ao mesmo calendário e ao mesmo "usuário criador" fixo. Isso precisa virar sessão real de usuário autenticado (o JWT resolve isso).
- Se a variável de ambiente do Supabase não estiver configurada, a página cai graciosamente para dados mock e continua funcionando (sem persistir nada).

4.3 Agenda do Professor (ProfessorAgendaPage)

- É uma **projeção da tabela** `eventos`, não uma entidade própria: busca eventos da semana do professor e classifica cada um em 4 tipos de atividade (`aula`, `planejamento`, `reuniao`, `evento`), a partir da categoria do evento.
- **Regra de "plano enviado":** uma atividade do tipo `aula` mostra o badge "Plano enviado" quando existe um `plano_aula` vinculado ao evento com `status` diferente de `rascunho`.
- **O upload de plano de aula é só uma modal visual — não grava nada.** É o primeiro caso de uso natural para o Supabase Storage decidido na seção 2.
- Grade de horários fixa mencionada na documentação original do design (07:30–16:20, com intervalos) — hoje a tela não usa `faixas_horario` do banco para desenhar a grade.

4.4 Painel da Direção (DirecaoPage)

- **Validação de planos de aula é o fluxo mais "real" desta tela:** lista `planos_aula` com status `enviado` ou `em_validacao`; aprovar/rejeitar atualiza o status para `aprovado` / `rejeitado` no banco. O status `revisao_solicitada` não tem nenhuma ação associada na UI hoje.
- **Eventos institucionais** exibidos usam uma lista fixa de categorias: `reuniao`, `pais`, `institucional`, `planejamento`, `formacao`, `conselho_classe`. Prioridade é derivada da categoria no frontend, não é um campo próprio.

- **"Criar Evento Institucional" e "Enviar Notificação Institucional" são modais 100% visuais — não persistem nada.** Os checkboxes de público-alvo já antecipam o campo `tipo_destinatario` (`usuario` / `papel` / `turma` / `todos`) do schema.
- Estatísticas do painel já vêm de contagens reais no Supabase.

4.5 Central de Notificações (NotificationsPage)

- Tipos: `reuniao`, `evento`, `validacao`, `atualizacao`, `lembrete`, `sistema`; prioridade `alta` / `media` / `baixa`.
- **"Lida"** hoje é calculado globalmente (todos os destinatários com `lida_em` preenchido), não por usuário, porque ainda não existe usuário autenticado. Deve virar "lida para o usuário logado".
- "Marcar todas como lidas" é real. **"Excluir notificação" é só local** — não apaga nem marca nada no banco.

4.6 Home / Dashboard (HomePage, DashboardPage)

- Estatísticas da equipe contam usuários por `papel`. Datas especiais/feriados vêm de `calendario_excecoes`.
- **Mural de avisos depende de uma tabela `avisos` que não existe no schema atual.** Precisa entrar via migration do Prisma se o mural for mantido no MVP.
- `DashboardPage` reaproveitado nas rotas placeholder `/dashboard/eventos` e `/dashboard/professores` parece ser uma versão mais antiga, possivelmente substituída pela `HomePage` — vale confirmar se ainda é necessário.

4.7 Configurações (SettingsPage)

Tela de preferências de notificação e tema — **toda em state local, nada é persistido.** O schema já tem a tabela `configuracoes_notificacao` praticamente pronta para receber esses dados.

---

5. Modelo de dados

A modelagem já existente em `banco-de-dados/dbml_prototipo_completo_escolaagenda.dbml` (364 linhas) é a fonte de verdade — é o schema que os nomes de tabela/coluna usados no frontend já pressupõem. Cobre:

- Estrutura institucional: `escolas`, `usuarios`, `perfis_profissionais`, `disciplinas`, `turmas`, `salas`, `professor_turma_disciplina`.
- Núcleo de agenda: `calendarios`, `faixas_horario`, `eventos`, `evento_participantes`, `eventos_excecao`, `calendario_excecoes`.
- Planos de aula: `planos_aula`, `anexos`, `plano_aula_anexos`, `validacoes_plano_aula` (o campo `storage_path` / `url_publica` de `anexos` passa a apontar para objetos no Supabase Storage).
- Notificações: `notificacoes`, `notificacao_destinatarios`, `configuracoes_notificacao`.

- Observabilidade: `metricas_dashboard`, `auditoria`.

Esse schema será traduzido para o `schema.prisma` do backend (migrations versionadas a partir daí), rodando contra o Postgres do Neon.

O outro arquivo (`dbml_planilhas_agenda_escolar.dbml`) é um rascunho anterior derivado direto das planilhas brutas em `arquivos/` — recomendo arquivá-lo ou marcá-lo claramente como histórico.

Lacunas identificadas no schema frente ao que o frontend já espera:

- Falta a tabela `avisos` (mural de avisos da Home).
- Não há `CHECK constraints` para `eventos.data_fim > eventos.data_inicio`, nem qualquer mecanismo de detecção de conflito de horário/sala.

---

6. Resumo dos gaps (o que existe só de "fachada" hoje)

Funcionalidade	Estado real
Login / autenticação	Simulado, sem validação de credenciais
Seleção de perfil	Não persiste, não bloqueia nenhuma rota
Guarda de rotas por papel	Inexistente — todas as rotas são públicas
Criar evento institucional (Direção)	Modal visual, não grava
Enviar notificação institucional (Direção)	Modal visual, não grava
Upload de plano de aula (Professor)	Modal visual, não grava nem faz upload
Excluir notificação	Só remove da tela, não do banco
Preferências de notificação (Configurações)	Só estado local
Mural de avisos (Home)	Depende de tabela inexistente no schema
Recuperação de senha	Só UI, sem envio de e-mail

O que já funciona de ponta a ponta com o Supabase hoje: leitura e criação de eventos (Agenda), leitura da agenda do professor, leitura/estatísticas da Direção, leitura e aprovação/rejeição de planos de aula, leitura e marcação de notificações como lidas, estatísticas da Home/Dashboard.

---

7. Escopo sugerido para a primeira fase do backend

Alinhado ao foco em agenda como núcleo do produto, sugiro que a API nova (Express + Prisma + Neon) cubra primeiro exatamente o que a Agenda e a Agenda do Professor já consomem, para poder

trocar o Supabase por ela sem regressão:

1. Autenticação real (login com JWT, papéis, guarda de rotas).
2. CRUD de `eventos` + `evento_participantes` (com validação de datas e, idealmente, checagem de conflito).
3. Leitura de `calendarios`, `turmas`, `salas`, `disciplinas`, `usuarios` (cadastros de apoio necessários para montar a agenda).
4. Fluxo de `planos_aula` (criar/enviar pelo professor com upload para o Supabase Storage, aprovar/rejeitar/solicitar revisão pela direção).
5. `notificacoes` + `notificacao_destinatarios` (criação real a partir de eventos institucionais, marcação de lida por usuário logado).

Deixaria para uma segunda fase: mural de avisos, métricas de dashboard, auditoria — nenhuma dessas bloqueia o uso do produto no dia a dia.

---

8. Análise de dados (Analytics)

Ponto trazido pela equipe: ter alguma forma de análise de dados no produto, com o critério de **simples e dinâmico**. Duas linhas possíveis — não são mutuamente exclusivas, a decisão de incluir uma, outra ou as duas fica com a equipe.

8.1 Opção 1 — Analytics de uso do produto

Métricas sobre o uso do próprio sistema (logins, telas mais acessadas, frequência de uso por perfil, etc.) — telemetria da aplicação, não do domínio escolar. Mais útil para o time entender adoção/engajamento do produto do que para a escola em si.

8.2 Opção 2 — Análise dos dados de domínio (recomendada)

Métricas sobre os dados que o sistema já gera no dia a dia da escola: eventos por categoria, taxa de aprovação de planos de aula, ocupação de salas, engajamento com notificações, carga de reuniões por professor. É o sentido de "análise de dados" mais alinhado ao produto (agenda escolar) e já tem base pronta no schema — a tabela `metricas_dashboard` foi desenhada exatamente para isso, hoje sem nenhuma lógica alimentando ela.

> **Como implementar de forma simples e dinâmica**, sem adicionar uma ferramenta de BI separada (Metabase/Superset), que exigiria hospedar mais um serviço:
>
> - **Backend:** endpoints de agregação usando Prisma (`groupBy`, `count`, `avg`) sobre `eventos`, `planos_aula`, `notificacoes`.
> - **Frontend:** usar `recharts`, que já está instalado no `prototipo/package.json` e não é usado em nenhuma tela hoje, para montar uma seção "Análises" dentro do Painel da Direção.

Isso mantém tudo dentro do mesmo backend/deploy já decidido (Render + Neon), sem infraestrutura extra.

---

9. Decisões pendentes para a equipe

1. Os papéis administrativos (`vice_diretor` , `secretario` , `administrador`) entram no MVP?
2. O mural de avisos entra no MVP (precisa de tabela nova) ou fica para depois?
3. O fluxo de "revisão solicitada" nos planos de aula (status `revisao_solicitada`) entra no MVP ou o painel da Direção continua só com aprovar/rejeitar?
4. Vincular participantes de evento a usuários/turmas reais (em vez de texto livre) entra no MVP da Agenda?
5. Analytics (seção 8): manter só a opção 2 (recomendação), incluir também a opção 1, ou não priorizar analytics agora?

Este documento não implica nenhuma alteração no código do repositório — é material de alinhamento para a equipe.